
Critérios de teste estrutural

Cobertura, grafos de fluxo e seleção de casos de teste



Prof. Dr. João Choma

Universidade Estadual de Maringá - DIN/CTC

github.com/JoaoChoma

Teste e Qualidade de Software



- Construir um grafo de fluxo de controle a partir de código simples.
- Selecionar entradas para cobrir instruções, ramificações e condições.
- Distinguir cobertura de caminhos de cobertura de ramificações.
- Projetar testes para ciclos e justificar limites de cobertura.
- Relacionar cada execução a um resultado esperado verificável.

Pré-requisitos: condicionais, operadores booleanos, laços e testes de unidade.



Teste estrutural (caixa-branca): seleciona testes com base na estrutura interna do programa.

- O código ajuda a decidir **por onde executar**.
- A especificação define **qual resultado esperar**: esse é o oráculo.
- Um teste contém entradas, preparação, execução e verificações.
- Uma falha aparece quando o resultado observado diverge do esperado.

Cobertura mede execução. Assertões verificam comportamento.



- **Critério:** regra usada para selecionar elementos que devem ser exercitados.
- **Requisito de teste:** elemento específico exigido pelo critério, como o ramo falso de um `if`.
- **Conjunto de testes:** execuções que satisfazem esses requisitos.

$$\text{Cobertura} = \frac{\text{requisitos exercitados}}{\text{requisitos considerados}} \times 100\%$$

Exemplo: três resultados de decisão exercitados entre quatro considerados dão **75% de cobertura**. Declare sempre o denominador e o escopo.



- **Nó:** instrução ou bloco de instruções executadas em sequência.
- **Aresta:** transferência possível de controle entre dois nós.
- **Decisão:** nó com saídas alternativas, rotuladas V e F nos exemplos.
- **Caminho:** sequência de nós conectados, da entrada à saída.
- **Ciclo:** percurso que permite retornar a um nó já visitado.

Um bloco pode agrupar várias linhas. O grafo representa a execução, não a formatação do arquivo.

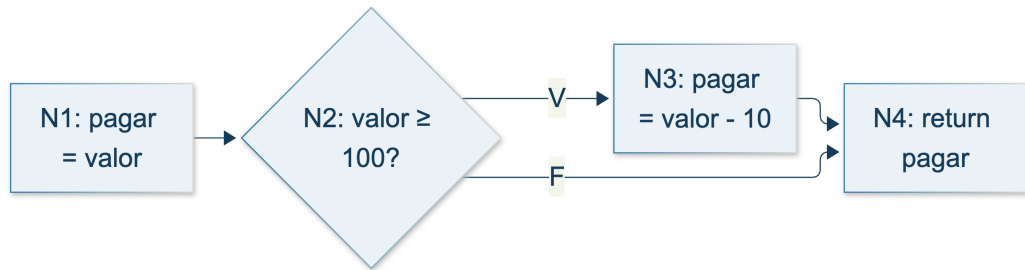
Exemplo 1: desconto por valor da compra



Regra: compras a partir de R\$ 100 recebem R\$ 10 de desconto. Considere valores inteiros não negativos.

```
static int total(int valor) {  
    int pagar = valor;           // N1  
    if (valor >= 100) {          // N2  
        pagar = valor - 10;      // N3  
    }  
    return pagar;                // N4  
}
```

Antes de calcular cobertura: qual deve ser o resultado para **99**, **100** e **150**?



V: N1, N2, N3, N4. **F:** N1, N2, N4. Entrada em N1 e saída após N4.



- **Linhas:** linhas executáveis atingidas conforme a definição da ferramenta.
- **Instruções:** comandos executáveis exercitados, independentemente de sua distribuição em linhas.
- Comentários, linhas vazias e chaves isoladas não são requisitos executáveis.
- Uma linha pode conter vários comandos e ficar apenas parcialmente coberta.

No desconto, contaremos **quatro nós executáveis**: N1, N2, N3 e N4. Essa é a convenção didática do exemplo, não um relatório de bytecode.



Todas as instruções com um único teste

Teste	Entrada	Esperado	Nós visitados
T1	150	140	N1, N2, N3, N4

- T1 cobre **4/4 nós = 100%** no modelo adotado.
- A decisão foi verdadeira, mas nunca foi falsa.
- O comportamento de uma compra sem desconto ainda não foi exercitado.

Pergunta: um teste com 150 detectaria trocar ≥ 100 por > 100 ?



Exige exercitar cada saída de decisão. Em um `if` binário, isso inclui V e F, mesmo quando não há `else` escrito.

Teste	Entrada	Esperado	Resultado de N2
T1	150	140	V
T2	99	99	F

- T1 sozinho: **1/2 saídas = 50%**.
- T1 e T2: **2/2 saídas = 100%**.
- Acrescente 100 com esperado 90 para verificar o limite da regra.

Aqui o denominador inclui apenas saídas condicionais. Algumas definições de cobertura de arestas incluem também ligações incondicionais.



```
// Implementacao com defeito:  
if (valor > 100) {  
    pagar = valor - 10;  
}
```

- Os testes 150 e 99 passam e cobrem V e F.
- O teste 100 deveria retornar **90**, mas retorna **100**.
- Cobertura estrutural deve ser combinada com análise de valores-limite e testes derivados da especificação.

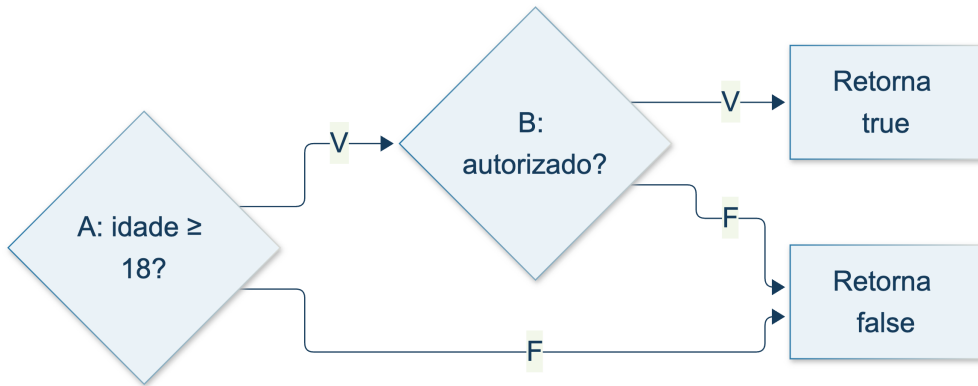
100% de cobertura não demonstra ausência de defeitos.



```
static boolean podeEntrar(int idade, boolean autorizado) {  
    return idade >= 18 && autorizado;  
}
```

- Condição A: idade >= 18.
- Condição B: autorizado.
- Decisão D: A && B.
- **Cobertura de condições:** cada condição atômica deve ser avaliada como V e F.
- **Cobertura de decisão:** o resultado composto deve assumir V e F.

Em Java, && e || usam curto-circuito. Um valor fornecido pode não ser avaliado.



Se A for F, B não será avaliada. O grafo explicita as decisões elementares realizadas durante a execução.



Conjunto que cobre as condições de A e B

Teste	Idade	Autorizado	A avaliada	B avaliada	D
C1	20	true	V	V	V
C2	20	false	V	F	F
C3	16	true	F	não	F

- A: V e F. B: V e F. **4/4 resultados atômicos cobertos.**
- D: V e F. **2/2 resultados da decisão composta cobertos.**
- C1 e C3 cobrem D, mas não exercitam B como F.

Registrar B como V em C3 seria confundir **valor de entrada** com **avaliação executada**.



Considere $A \ || \ B$ com curto-circuito:

Teste	A	B avaliada	Decisão
Q1	V	não	V
Q2	F	F	F

Há 100% dos resultados da decisão, mas falta avaliar B como V.

No sentido inverso, com avaliação completa de $A \ | \ B$ booleano em Java, os pares (V,F) e (F,V) cobrem V/F de A e B, mas a decisão é sempre V.

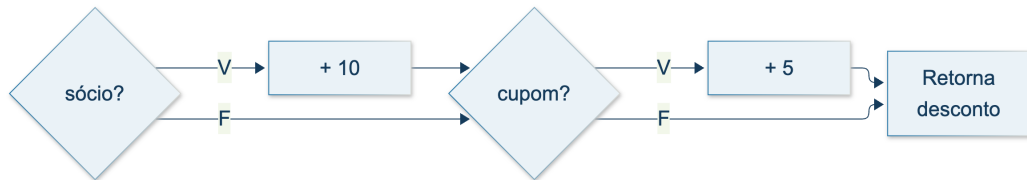
Sempre declare o operador e a semântica de avaliação.



Exige percorrer as sequências completas de execução consideradas no grafo.

```
static int bonus(boolean socio, boolean cupom) {  
    int desconto = 0;  
    if (socio) desconto += 10;  
    if (cupom) desconto += 5;  
    return desconto;  
}
```

As decisões são independentes. Há quatro caminhos: **VV**, **VF**, **FV** e **FF**, com retornos 15, 10, 5 e 0.



Na entrada, **desconto** = 0. Os caminhos se reencontram antes de testar cupom. Cobrir saídas isoladas não cobre todas as combinações.



Teste	Sócio	Cupom	Retorno	Caminho
P1	true	true	15	VV
P2	false	false	0	FF

- Saídas condicionais: V e F nas duas decisões, $4/4 = 100\%$.
- Caminhos completos: apenas VV e FF, $2/4 = 50\%$.
- Adicione (true, false) e (false, true) para cobrir os quatro.

Com k decisões binárias independentes em sequência, podem existir 2^k caminhos. Para $k = 10$, são **1.024**.



```
if (x > 0) { /* bloco A */ }  
if (x < 0) { /* bloco B */ }
```

- Sem alteração de x entre decisões, o caminho VV é inviável.
- Justifique inviabilidade pelas restrições de entrada, não pela dificuldade de encontrar um teste.
- Laços sem limite fixo podem gerar infinitos caminhos.
- Quando a cobertura integral é inviável, delimite os requisitos e registre os não cobertos e suas razões.

Não remova requisitos do denominador apenas para elevar o percentual.



Critério	O que exige	Limitação principal
Instruções	Executar cada comando	Pode ignorar uma saída
Ramificações	Exercitar cada saída	Pode omitir combinações
Condições	V e F por condição	Não equivale a decisão
Caminhos	Sequências completas	Explosão e inviabilidade
Ciclos	Contagens e fronteiras	Depende dos limites

No mesmo grafo e escopo, cobrir todas as arestas cobre os nós alcançáveis. Isso não estabelece uma ordem universal entre todos os critérios e métricas de ferramentas.



- **Simples:** um único laço, sem outro em seu corpo.
- **Aninhados:** um laço dentro de outro.
- **Concatenados:** laços executados em sequência.
- **Não estruturados:** saltos que produzem entradas ou retornos irregulares.

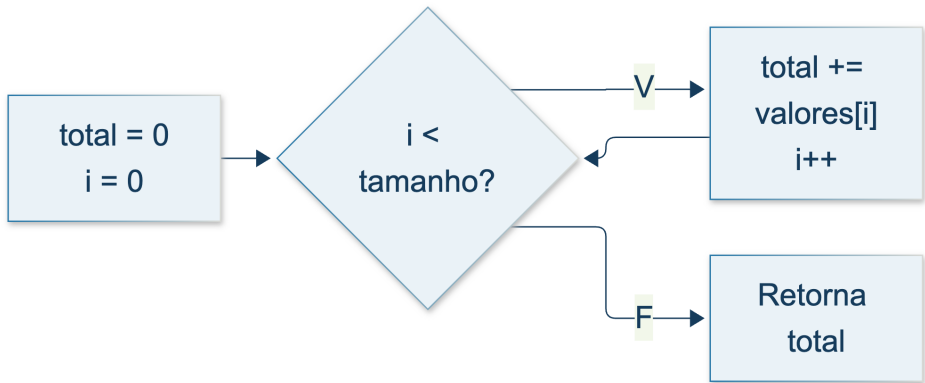
Testar ciclos envolve número de iterações, condições de entrada e saída, atualização do estado e resultado produzido.

Exemplo 2: soma de uma lista



Contrato: lista de inteiros não nula, com até cinco elementos pequenos. Lista vazia produz zero. Mais de cinco elementos deve ser rejeitado.

```
static int soma(int[] valores) {  
    if (valores.length > 5)  
        throw new IllegalArgumentException();  
    int total = 0;  
    int i = 0;  
    while (i < valores.length) {  
        total += valores[i];  
        i++;  
    }  
    return total;  
}
```



O grafo mostra o trecho **após a validação**. A rejeição por tamanho também precisa de teste, mas está fora deste recorte.



Testes de um ciclo simples

Para limite $n = 5$, use as contagens 0, 1, 2, m , $n-1$, n e uma entrada que solicite $n+1$, com $2 < m < n-1$ quando possível.

Tamanho	Entrada	Esperado
0	[]	0
1	[2]	2
2	[2, 3]	5
3 (m)	[1, 2, 3]	6
4 ($n-1$)	[1, 2, 3, 4]	10
5 (n)	[1, 2, 3, 4, 5]	15
6 ($n+1$)	[1, 2, 3, 4, 5, 6]	Exceção

O caso $n+1$ valida rejeição: **não exige executar seis iterações.**



O que os testes do ciclo procuram

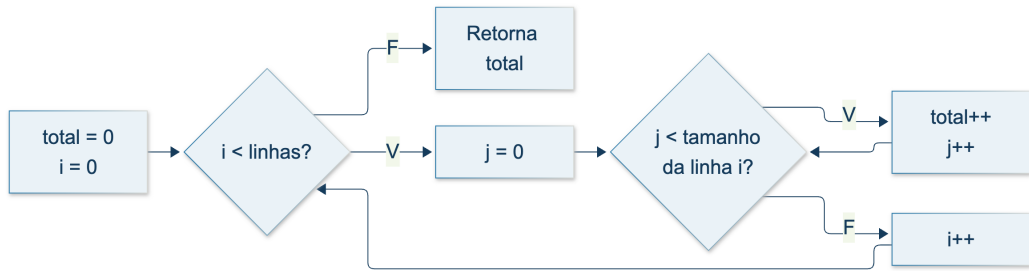
- **Zero:** inicialização correta e possibilidade de pular o corpo.
- **Uma e duas:** primeira execução e atualização para a seguinte.
- **Valores intermediários:** acumulação e comportamento repetido.
- **Fronteiras:** erros como $<$ trocado por $\leq$.
- **Além do limite:** rejeição ou tratamento definido pelo contrato.

Um `do...while` sempre executa o corpo ao menos uma vez. Zero iterações pode ser inviável. Testes repetidos não provam terminação para todas as entradas.



```
static int contar(int[] [] matriz) {  
    int total = 0;  
    for (int i = 0; i < matriz.length; i++) {  
        for (int j = 0; j < matriz[i].length; j++) {  
            total++;  
        }  
    }  
    return total;  
}
```

Considere matriz e linhas não nulas. O resultado esperado é a soma dos comprimentos das linhas, inclusive quando eles diferem.



`j` reinicia a cada linha. A saída do ciclo interno atualiza `i` e retorna ao teste do ciclo externo.



- 1 Comece pelo laço mais interno.
- 2 Mantenha os externos no mínimo que permita alcançar o interno.
- 3 Varie as contagens do interno como em um ciclo simples.
- 4 Avance para o próximo laço externo, fixando os internos em valores representativos.
- 5 Acrescente fronteiras, entradas inválidas e interações relevantes.

Atenção: fixar o externo em zero impede testar o interno. Essa estratégia reduz combinações, mas não garante cobertura de todos os caminhos.



Matriz	Iterações por linha	Esperado	Objetivo
[]	nenhuma	0	Pular o externo
[[]]	0	0	Pular o interno
[[7]]	1	1	Uma execução
[[7,8]]	2	2	Repetir o interno
[[7],[8]]	1 e 1	2	Reiniciar j
[[7,8],[],[9]]	2, 0 e 1	3	Interação de tamanhos

Se a aplicação limitar linhas e colunas, adicione testes nas respectivas fronteiras e acima delas, conforme o contrato.



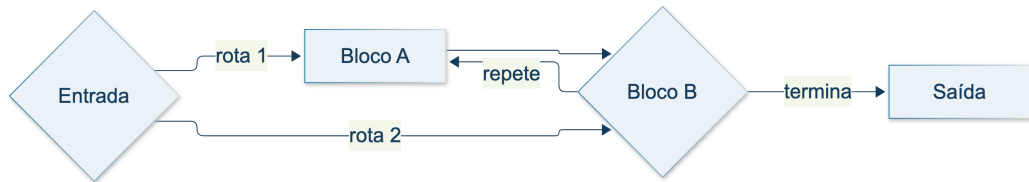
```
for (int i = 0; i < a.length; i++) {  
    processarA(a[i]);  
}  
for (int j = 0; j < b.length; j++) {  
    processarB(b[j]);  
}
```

- Se dados, limites e efeitos não conectam os ciclos, teste cada um como ciclo simples.
- Inclua ambos vazios, somente A vazio, somente B vazio e ambos não vazios.
- Verifique os resultados de cada processamento, não apenas suas contagens.



```
int positivos = 0;
for (int x : valores) {
    if (x > 0) positivos++;
}
for (int j = 0; j < positivos; j++) {
    emitirAviso();
}
```

- O primeiro laço determina o limite do segundo.
- 0, -1 , [2, 0] e [2, 3] devem produzir **0, 1 e 2 avisos**.
- Varie o resultado do produtor e confira o comportamento do consumidor.
 - Use uma estratégia coordenada, como nos aninhados, em vez de presumir independência.



A região A/B tem duas entradas externas. Quando possível, reorganize o fluxo com construções estruturadas e preserve o comportamento com testes de regressão.



Escopo	Exemplo de verificação
Método	total(100) retorna 90
Classe	depositar e sacar preservam o saldo
Integração	Pedido usa Estoque e Pagamento corretamente
Sistema	Compra completa produz pedido e comprovante

- Cobertura estrutural é um critério de seleção, não um nível de teste.
- O teste de classe enfatiza estado e interação entre métodos.
- A classificação de classe como unidade ou componente depende da organização do sistema.



- Identifique revisão do código, conjunto executado e escopo medido.
- Procure linhas e saídas não exercitadas e derive novas entradas.
- Confira exclusões, código gerado e diferenças entre fonte e bytecode.
- Verifique se os testes têm asserções úteis e oráculos corretos.

Exemplo de ferramenta: no JaCoCo, linhas, instruções de bytecode e branches são contadores distintos. O contador de branches não inclui tratamento de exceções.

Fonte: documentação oficial do JaCoCo, *Coverage Counters*.



```
static int tarifa(int idade, boolean estudante) {  
    int valor = 20;  
    if (idade < 12) valor = 0;  
    if (estudante) valor = valor / 2;  
    return valor;  
}
```

- 1 Desenhe o grafo e enumere os caminhos completos.
- 2 Escolha dois testes para 100% das saídas condicionais.
- 3 Complete a suíte para cobrir todos os caminhos.
- 4 Declare o retorno esperado e acrescente testes de fronteira.

Regra: menores de 12 não pagam, e estudantes recebem metade da tarifa resultante.
Considere idades não negativas.



Idade	Estudante	Caminho	Retorno
11	true	VV	0
12	false	FF	20
11	false	VF	0
12	true	FV	10

- Os dois primeiros testes cobrem as quatro saídas condicionais.
- Os quatro testes cobrem os quatro caminhos viáveis.
- As idades 11 e 12 já exercitam a fronteira do predicado.
- VV e VF têm o mesmo retorno, mas percorrem estruturas diferentes.



Uma suíte testa soma apenas com [1] e [1, 2, 3, 4, 5].

- 1 A condição do `while` assume V e F?
- 2 Quais contagens e comportamentos ficaram sem teste?
- 3 Trocar `total += valores[i]` por `total = valores[i]` seria detectado?
- 4 Que asserções devem acompanhar os casos vazio e acima do limite?

Discussão: diferencie um requisito estrutural coberto de uma obrigação funcional verificada.



- O `while` assume `V` ao entrar e `F` ao terminar. Seus dois resultados estão cobertos.
- Faltam zero, duas, uma contagem intermediária e $n-1$ iterações, além da rejeição de tamanho 6.
- A atribuição incorreta retorna 5 para `[1,2,3,4,5]`, em vez de 15. A asserção do resultado detecta o defeito.
- Para `[]`, verifique retorno 0.
- Para seis elementos, verifique `IllegalArgumentException`.

Uma execução sem asserção pode aumentar a cobertura e ainda deixar passar um resultado incorreto.



- 1 Leia o contrato e identifique entradas válidas, inválidas e fronteiras.
- 2 Modele o fluxo e selecione o critério conforme o risco.
- 3 Liste requisitos de teste e encontre entradas que os exercitem.
- 4 Registre os resultados esperados antes de executar.
- 5 Execute, meça a cobertura e investigue lacunas.
- 6 Documente inviabilidades e combine critérios estruturais e funcionais.

Entrega de um teste: entrada + caminho pretendido + esperado + observado + conclusão.



Desafio: implementar e testar dois algoritmos para o mesmo contrato.

- **Insertion Sort:** estudar o laço externo e os deslocamentos do laço interno.
- **Merge Sort:** estudar divisão recursiva e os laços de intercalação.
- **Em sala (50 min):** simular, desenhar os grafos e planejar os testes, em duplas ou trios.
- **No laboratório (80 a 100 min):** implementar e executar o plano da aula anterior.

Os algoritmos-base são fornecidos. O grupo deve projetar **como demonstrar que funcionam** e quais estruturas os testes exercitam.



- Receber um vetor de inteiros e devolver uma **nova cópia ordenada em ordem não decrescente**.
- Preservar o vetor original, o tamanho e a quantidade de ocorrências de cada valor.
- Aceitar vetor vazio, um elemento, negativos, zero e repetidos.
- Para entrada nula, lançar `IllegalArgumentException` antes de chamar o algoritmo-base.

Exemplo: $[3, -1, 3, 0]$ deve produzir $[-1, 0, 3, 3]$.

Testes manuais: vetores de até 6 elementos, por praticidade. **Seis não é um limite do programa;** um vetor de tamanho 7 também deve funcionar.



```
INSERTION(v):  
  a = copia(v)  
  para i de 1 ate tamanho(a)-1:  
    chave = a[i]  
    j = i-1  
    enquanto j >= 0 E a[j] > chave:  
      a[j+1] = a[j]  
      j = j-1  
    a[j+1] = chave  
  retorne a
```

Ideia: inserir a chave na parte que já está ordenada.

Avaliar E da esquerda para a direita, com curto-circuito. Se $j < 0$, não acessar $a[j]$.



Simulação e requisitos do Insertion Sort

- Simular $[3, 1, 2]$: registrar i , chave, j , cada deslocamento e o vetor ao final de cada passagem externa.
- Para tamanho n , o externo executa $\max(0, n - 1)$ vezes.
- Na passagem i , o interno pode executar de **0 a i** deslocamentos.
- Projetar casos com zero, um e vários deslocamentos, incluindo chegar ao início do vetor.
- Distinguir saídas do interno: $j < 0$ e $a[j] \leq \text{chave}$.

Pergunta: vetores do mesmo tamanho, um crescente e outro decrescente, exercitam os laços da mesma maneira?



```
MERGE(v):  
    se tamanho(v) <= 1:  
        retorne copia(v)  
    meio = tamanho(v) / 2  // divisao inteira  
    esquerda = MERGE(v[0:meio])  
    direita = MERGE(v[meio:tamanho(v)])  
    retorne INTERCALAR(esquerda, direita)
```

`v[inicio:fim]` inclui inicio e exclui fim. Cada fatia é uma cópia.

Ideia: ordenar as metades e intercalá-las. A recursão reduz o problema; os laços ficam na intercalação.

Na simulação, incluir tamanhos **par e ímpar** e identificar o caso-base.



```
INTERCALAR(e, d): // e e d ja estao ordenados
  r = lista vazia; i = 0; j = 0
  enquanto i < tamanho(e) E j < tamanho(d):
    se e[i] <= d[j]:
      adicionar e[i] a r; i = i+1
    senao:
      adicionar d[j] a r; j = j+1
  enquanto i < tamanho(e):
    adicionar e[i] a r; i = i+1
  enquanto j < tamanho(d):
    adicionar d[j] a r; j = j+1
  retorne r como vetor
```

O primeiro laço compara valores. Os outros copiam o restante.

Investigar: quando cada laço de restante executa zero vezes?



- 1 Ler o contrato e simular os dois algoritmos em $[3, 1, 2]$ (10 min).
- 2 Desenhar um grafo para Insertion Sort e outro para INTERCALAR; representar a recursão por uma árvore de chamadas (15 min).
- 3 Selecionar pelo menos **8 entradas comuns**, calcular a saída e relacionar cada caso aos requisitos estruturais (20 min).
- 4 Revisar o plano com outra dupla, sem executar código (5 min).

Cada caso: ID, entrada, saída esperada ou exceção, objetivo, caminho previsto e contagens dos laços. Registrar separadamente as contagens de cada algoritmo.



Casos que a suíte deve contemplar

- Vazio e um elemento; dois elementos em ordem e em ordem inversa.
- Vetores crescente e decrescente com pelo menos 4 elementos.
- Todos os valores iguais; mistura de negativos, zero e repetidos.
- Tamanhos par e ímpar; entrada nula com exceção esperada.
- No Merge Sort: escolher valores que esgotem primeiro a esquerda e outros que esgotem primeiro a direita; incluir empate.

Um caso pode atender a vários requisitos. O mínimo de 8 entradas **não dispensa os demais casos necessários**.

Para testar INTERCALAR isoladamente, fornecer metades já ordenadas e incluir uma metade vazia.



- 1 Implementar os dois algoritmos em Java, seguindo o pseudocódigo e validando a entrada nula.
- 2 Automatizar a mesma suíte para ambos, com JUnit ou verificações executáveis equivalentes.
- 3 Verificar saída completa, nova cópia e preservação da entrada. Não usar ordenação pronta na implementação.
- 4 Executar o plano original; registrar resultados por ID. Corrigir divergências e acrescentar casos para lacunas.
- 5 Conferir caminhos e iterações com depurador ou contadores temporários. Preservar as previsões originais.

Ordenação pronta pode ser usada **apenas nos testes**, sobre uma cópia, como oráculo adicional para vetores maiores.



- Insertion Sort: contar passagens externas, deslocamentos por passagem e comparações $a[j] > \text{chave}$ efetivamente avaliadas.
- Merge Sort: contar chamadas, iterações de cada intercalação e comparações $e[i] \leq d[j]$.
- Para comparar comparações de valores, excluir testes de índices, validações e condições de parada.
- Rodar vetores crescente, decrescente e com repetidos, sempre com cópias novas da mesma entrada.

Apresentar contagens e explicar diferenças. Poucos vetores pequenos **não demonstram o desempenho geral** de um algoritmo.



Introduzir **um defeito por vez** na versão correta:

- ① Insertion Sort: trocar $j \geq 0$ por $j > 0$.
- ② Insertion Sort: retirar a atribuição final $a[j+1] = chave$.
- ③ Merge Sort: retirar o laço que copia o restante da direita.

Identificar o teste que detecta cada defeito e a divergência observada. Restaurar a versão correta antes do próximo experimento.

Se nenhum teste falhar, acrescentar um caso e explicar a lacuna. Apenas comparar os dois algoritmos **não substitui um oráculo correto**.



Entregar: simulações, grafos, árvore de chamadas, plano original, duas implementações, testes e registro de execução.

- simulação, modelagem e requisitos de teste.
- diversidade das entradas e oráculos corretos.
- implementação, execução reproduzível e correções.
- análise dos defeitos, contagens e limites da cobertura.

Discussão final: por que duas execuções que ordenam corretamente podem exercitar os laços de maneiras diferentes?



- **JaCoCo.** *Coverage Counters*. Documentação oficial dos contadores de cobertura.
- **Beizer, Boris.** *Software Testing Techniques*. 2ª ed., 1990. Referência de ciclos mencionada no material-base.